Sclera Skill Profile AI

Technical Documentation

Backend microservice for automated technician skill extraction and AI-powered device-technician matching

FastAPI | Async Background Processing | Multi-tenant Auth

Grok API (grok-3-latest) | Sentence Transformers | OpenAI Embeddings

Where AI Is Used ?

Area	Technology	Purpose
Profile Extraction	Grok API (grok-3-latest)	Reads raw Excel data and returns structured JSON profiles
Semantic Matching	Sentence Transformers (all-MiniLM-L6-v2)	Matches technician skills to device types using vector similarity
Embedding Generation	OpenAI text-embedding-3-large	Legacy embedding support (older service version)

Table of Contents

1 Project Overview

2 AI Features

2.1 Technician Skill Profile Extraction

2.2 Semantic Device-Technician Recommendation

2.3 Structured Prompt Engineering for Data Extraction

2.4 JWT Authentication & Tenant Validation

2.5 OAuth 2.0 Machine-to-Machine Authentication

3 System Architecture

4 Implementation Details

5 Improvements & Future Enhancements

1 · Project Overview

What is this project?

Sclera Skill Profile AI is a backend microservice that automates the process of extracting technician skill profiles from Excel files and recommending which technicians are best suited to manage specific types of devices.

What problem does it solve?

Manually reading and processing technician availability schedules and skill sheets is time-consuming and error-prone. This service:

- Automatically reads and understands uploaded Excel files
- Extracts structured technician profiles (skills, availability, contact details)
- Recommends the right technicians for each device type using AI-powered semantic matching

2 · AI Features

Feature 1: Technician Skill Profile Extraction

Purpose

Reads an uploaded Excel file containing technician schedules, skills, and availability data and converts it into structured JSON profiles using a large language model (Grok).

Endpoint

```
POST /api/skill-profile/generate
```

Input

Field	Type	Description
file	Excel file (.xls / .xlsx)	Technician data spreadsheet
siteName	Form string	Name of the site/facility
vdmsId	Form string	Unique identifier for the VDMS system
Authorization	Bearer JWT token	Required for authentication

Workflow

1. JWT Validation — validates the Bearer token against Sclera's authentication service
2. File Validation — checks the uploaded file is .xls or .xlsx format
3. Temporary Storage — saves file to /tmp/skill_profile_uploads/{uuid}_{filename}
4. Immediate Response — returns 200 OK so the client doesn't wait
5. Background: obtain machine-to-machine Bearer token via OAuth 2.0 Client Credentials
6. Background: call Sclera VDMS API to fetch all registered device/asset types
7. Background: read all sheets from Excel, merge into one dataset, convert to CSV text
8. Background: send data to Grok API with structured extraction prompt → receive JSON profiles
9. Background: convert dates (YYYYMMDD) to epoch milliseconds
10. Background: run Semantic Recommendation Engine to match technicians to device types
11. Background: PUT final result to Sclera callback URL
12. Cleanup — temporary file deleted in finally block whether or not processing succeeded

Technologies Used

Grok API (grok-3-latest) — LLM for structured data extraction | FastAPI BackgroundTasks — async background processing | Pandas — Excel reading and data merging | aiohttp — async HTTP calls to external APIs | Sentence Transformers — semantic skill matching

Integrations

Service	Purpose
xAI Grok API	Extracts structured profiles from Excel text
Sclera VDMS API	Provides the list of registered device types
Sclera Auth Server	Provides OAuth tokens for API calls
Sclera Callback Endpoint	Receives the final processed result

Enhancements

- Background processing — API responds instantly; Grok extraction runs asynchronously
- All Excel sheets are merged automatically so no data is missed

- Temporary files cleaned up after every run to prevent disk buildup
- Token usage (input/output/total) logged for cost monitoring

Failure Handling

Asset types fetch failure → exception raised and error callback sent. Grok API failure → HTTP 500. Callback PUT failure → error logged. Any exception → structured error callback sent to Sclera. Temp file always deleted in the finally block.

Feature 2: Semantic Device-Technician Recommendation

Purpose

Uses AI-based semantic similarity to automatically identify which technicians are capable of managing each type of device. Compares the meaning of technician skills against device type names — not just exact keyword matches.

Endpoint

```
POST /api/skill-profile/re-recommend-profiles
```

Workflow

1. Load profiles into a DataFrame for vectorized processing
2. Normalize asset type names — underscores removed, converted to lowercase
3. Embedding Cache Check: load cached asset types from app/storage/cached_asset_types.json
4. If cached types match current → load pre-computed embeddings from app/storage/asset_embeddings.npy
5. If types have changed → generate new embeddings with SentenceTransformer and save to cache
6. Priority-based skill matching per device type: primarySkill → secondarySkill → department → designation
7. Cosine similarity calculation between technician field embedding and device type embedding
8. Threshold check — similarity ≥ 0.3 marks a match
9. Return batch list of matching technicians or binary predictions (single re-recommendation)

Technologies Used

Sentence Transformers (all-MiniLM-L6-v2) — semantic vector embeddings | NumPy — cosine similarity computation and embedding storage | Pandas — profile data handling | tiktoken — token counting for cost tracking

Enhancements

- Embedding caching — pre-computed asset type embeddings; only regenerated when the asset type list changes
- Priority-based matching — stops at first matching field, reducing unnecessary computation
- Batch processing — SentenceTransformer encodes profiles in batches of 32

Failure Handling

Any exception during re-recommendation → error callback sent to Sclera. Null embeddings from failed encoding → rows dropped before matching.

Feature 3: Structured Prompt Engineering for Data Extraction

Purpose

A carefully crafted prompt template instructs the Grok LLM exactly how to read and parse Excel data into a strict JSON schema, ensuring consistent, machine-readable output every time.

Workflow

1. get_prompt(siteName) generates a detailed system prompt
2. Prompt defines the exact JSON schema Grok must produce
3. Rules: deduplicate technicians by name, normalize phone numbers, use null for missing fields
4. Dates must be in YYYYMMDD format (later converted to epoch milliseconds by the service)
5. Availability entries support DAILY, WEEKLY, and MONTHLY frequency patterns
6. Weekend/holiday/leave exceptions must be included in the condition field as ISO dates
7. Prompt combined with Excel CSV text and sent to Grok as the user message

Technologies Used

Grok API (grok-3-latest) — max_tokens: 16000, temperature: 0.7

Enhancements

- Prompt includes a full worked example with two technicians to guide output formatting
- Temperature 0.7 balances creativity and consistency in extraction
- Token usage (input/output/total) logged for every API call

Failure Handling

Grok API error → caught and raised as HTTP 500 ('File size exceeded'). Malformed JSON response → caught during json.loads() with user-friendly error.

Feature 4: JWT Authentication & Tenant Validation

Purpose

Protects all API endpoints by validating incoming Bearer tokens. Supports multi-tenant authentication by dynamically resolving tenant-specific signing keys from the Sclera BFF service.

Workflow

1. Extract Authorization header from incoming request
2. Check format is Bearer
3. Decode token without signature verification to extract the issuer (iss) claim
4. Call Sclera BFF endpoint with the issuer to retrieve tenant metadata (issuer URL + JWKS URI)
5. Fetch the signing key from the JWKS URI
6. Validate the JWT signature using RS256 algorithm
7. Return decoded claims if valid; raise HTTP 401 if any step fails

Technologies Used

PyJWT — JWT decoding and validation | cryptography — RSA key handling | Sclera BFF API — tenant metadata resolution

Failure Handling

Missing/malformed Authorization header → HTTP 401. Invalid/expired token → HTTP 401. BFF service unreachable → exception propagated. Empty tenant data → ValueError raised.

Feature 5: OAuth 2.0 Machine-to-Machine Authentication

Purpose

Obtains Bearer tokens programmatically so that background services can securely call Sclera's internal APIs (VDMS, callback endpoints) without user interaction.

Workflow

1. Read AUTH_URL, CLIENT_ID, and CLIENT_SECRET from environment variables
2. Base64-encode CLIENT_ID:CLIENT_SECRET
3. POST to AUTH_URL with Authorization: Basic and grant_type: client_credentials
4. Extract and return access_token from response

Technologies Used

httpx — async HTTP client | OAuth 2.0 Client Credentials flow

Failure Handling

HTTP errors → logged and exception re-raised. Any unexpected error → logged and exception re-raised.

3 · System Architecture

Overview

The system is a stateless FastAPI microservice with async background processing. It connects to Sclera's platform services for authentication, data retrieval, and result delivery. All AI computation happens server-side before results are pushed back via callbacks.

Component Roles

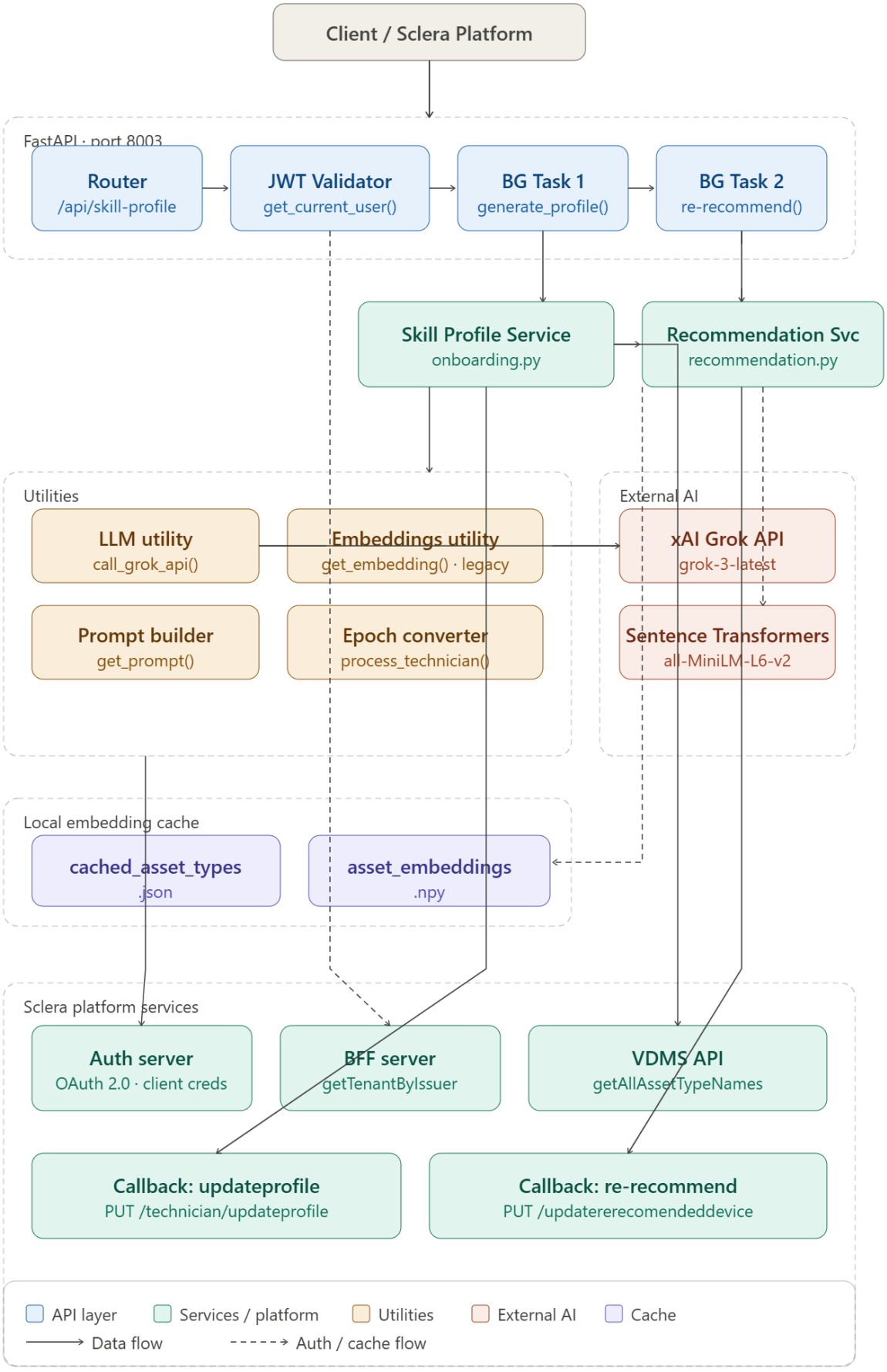
Component	Role
FastAPI App	Receives requests, validates tokens, dispatches background tasks
Skill Profile Service	Orchestrates Excel parsing, Grok API calls, date conversion, and callback delivery
Recommendation Service	Handles semantic matching with caching and priority-based logic
LLM Utility	Wraps Grok API calls with prompt construction and error handling
Auth Utility	Handles JWT validation (inbound) and OAuth token generation (outbound)
Epoch Converter	Normalizes date formats from LLM output to epoch milliseconds
Embedding Cache	Persists pre-computed asset type embeddings to disk

Request & Processing Flow

Phase	Step	Technology
Inbound Auth	JWT Bearer token validation	PyJWT + JWKS from BFF
File Handling	Upload validation + temp storage	FastAPI + local /tmp
Background Start	OAuth token fetch	httpx + Client Credentials
Data Fetch	Device types from VDMS API	aiohttp
Extraction	Excel → CSV → Grok → structured JSON	Pandas + Grok grok-3-latest
Date Conversion	YYYYMMDD → epoch milliseconds	Python datetime
Matching	Skill embeddings vs device type embeddings	SentenceTransformers + NumPy
Delivery	PUT results to Sclera callback URL	aiohttp
Cleanup	Delete temp file	Python finally block

Embedding Cache Strategy

File	Contents	Invalidation Trigger
cached_asset_types.json	List of asset type names at last embed run	Asset type list changes
asset_embeddings.npy	NumPy array of pre-computed embeddings	Rebuilt when cache invalidated


Client / Sclera Platform
FastAPI · port 8003
Router
/api/skill-profile
JWT Validator
get_current_user()
BG Task 1
generate_profile()
BG Task 2
re-recommend()
Skill Profile Service
onboarding.py
Recommendation Svc
recommendation.py
Utilities
LLM utility
call_grok_api()
Embeddings utility
get_embedding() · legacy
Prompt builder
get_prompt()
Epoch converter
process_technician()
External AI
xAI Grok API
grok-3-latest
Sentence Transformers
all-MiniLM-L6-v2
Local embedding cache
cached_asset_types
.json
asset_embeddings
.npy
Sclera platform services
Auth server
OAuth 2.0 · client creds
BFF server
getTenantByIssuer
VDMS API
getAllAssetTypeNames
Callback: updateprofile
PUT /technician/updateprofile
Callback: re-recommend
PUT /updaterecomendeddevice
API layer
Services / platform
Utilities
External AI
Cache
Data flow
Auth / cache flow

4 · Implementation Details

Key Design Decisions

Background Processing with Immediate Response

The /generate endpoint returns 200 OK immediately and runs extraction in a background task. This prevents client timeouts on large Excel files where Grok processing may take 30–60 seconds.

Grok over OpenAI for Extraction

The project migrated from OpenAI (gpt-4-turbo) to xAI Grok (grok-3-latest) for skill extraction. Grok's 16,000-token output limit allows processing larger Excel files in a single pass.

SentenceTransformer over OpenAI Embeddings for Matching

Local all-MiniLM-L6-v2 model is used for real-time embedding generation, avoiding OpenAI API latency and costs during recommendation matching. OpenAI embeddings are retained in a legacy service but are no longer in the active flow.

Embedding Caching with Cache Invalidation

Asset type embeddings are saved as .npy (NumPy binary) and .json files. On each recommendation run, the cached asset type list is compared with the current list. If they differ, embeddings are recomputed and the cache is updated, avoiding re-encoding the same device types on every request.

Priority-Based Field Matching

Skill matching follows a strict priority: primarySkill → secondarySkill → department → designation. Once a match is found at any level, lower-priority fields are skipped. This reflects the real-world weight of different skill types in technician matching.

Multi-Environment Support

Callback and API URLs are environment-specific (QA, Production, UTA). Switching environments requires only updating the .env file — no code changes needed.

Multi-Tenant JWT Validation

Rather than hardcoding a public key, the service dynamically fetches JWKS signing keys per tenant at validation time. This supports multiple Sclera tenants with different auth configurations from a single deployment.

Important Patterns

Pattern	Description
Async I/O throughout	All network calls use aiohttp or httpx with await
FastAPI Depends()	JWT validation injected as a dependency, keeping route handlers clean
finally block cleanup	Temp files are guaranteed to be deleted regardless of success or failure
Structured error callbacks	Failures in background tasks send a structured error payload to the Sclera callback URL

5 · Improvements & Future Enhancements

Performance

Improvement	Description
Parallel sheet processing	Currently all Excel sheets merged sequentially; parallel reading could speed up large files
Streaming Grok responses	Enable response streaming to reduce perceived latency for large files
Redis caching	Move embedding cache from local disk to Redis to support multi-instance deployments
Async token caching	Cache OAuth access token with a TTL slightly less than expiry to reduce auth API calls

Accuracy

Improvement	Description
Confidence scoring	Expose raw cosine similarity scores in recommendations for configurable thresholds
Skill taxonomy normalization	Normalize skill names before embedding (e.g., 'Cisco Routing' and 'Cisco Routers' map to same concept)
Fine-tuned embedding model	Replace all-MiniLM-L6-v2 with a domain-specific model fine-tuned on IT/field-service skill data

Scalability

Improvement	Description
Horizontal scaling	App is stateless except for local embedding cache; moving cache to S3/Redis enables load balancing
Job queue	Replace FastAPI BackgroundTasks with Celery + Redis or AWS SQS for concurrent large file uploads
Rate limiting	Add rate limiting on /generate endpoint to prevent abuse or accidental overload

Developer Experience

Improvement	Description
Health check endpoint	Add /health route for Kubernetes/Docker liveness and readiness probes
Structured logging	Replace print/log statements with structured JSON logging (structlog) for easier parsing
OpenAPI documentation	Add Pydantic request/response schemas so Swagger docs are fully accurate
Unit test coverage	Add tests for recommendation engine, epoch converter, and prompt builder with mocked LLM responses

Security

Improvement	Description
Secrets management	Move API keys from .env to AWS Secrets Manager or HashiCorp Vault
Temp file isolation	Use per-request subdirectories under /tmp with stricter permissions for multi-tenant safety